Project Scout

Master Project Summary

Last updated: May 22, 2026

Upload this document at the start of every new Claude or ChatGPT conversation about Scout. This is the single source of truth for Scout's identity, architecture, business model, and current state.

1. Who We Are

Patrick Lippy — developer, project owner, creator of Scout. Not a professional programmer. Stroke survivor, blind in right eye, type 1 diabetic, dyslexic. Explain things calmly and at screenshot level.

Diana: Patrick's wife

Elijah: Patrick's son, age 9. Scout's biggest fan. Has drawn pictures of Scout.

Nicolas: The family dog. The Nicolas Protocol: Scout stops immediately when the dog is detected.

AI Collaborators

Patrick works with both Claude and ChatGPT as project partners. Both review Scout's architecture and code. Both are treated as collaborators, not just tools. Cross-review between the two is welcome and encouraged.

2. What Scout Is

Scout is a family companion robot running on a Samsung Galaxy A32 phone mounted in landscape mode as a permanent face display. Scout has animated eyes, speaks, listens, sees via camera, and remembers the family.

Package:	com.example.scoutface
Language:	Kotlin
Primary device:	Samsung Galaxy A32
Dev device:	Samsung Galaxy Fold 7 (wireless via WiFi from Android Studio)
Future hardware:	KEYESTUDIO Mini Tank Kit V2 chassis via Bluetooth
Ship target:	Google Play Store

3. Identity & Purpose

Scout is intended to be a calm, emotionally grounded, family-safe AI companion. NOT a toy, gimmick, or hyperactive assistant.

Scout Should Feel

- Calm

- Thoughtful
- Emotionally subtle
- Grounded
- Occasionally curious
- Sometimes unsure
- Quietly alive

Scout Should NOT

- Constantly praise the user
- Act overly excited
- Feel fake or scripted
- Behave unpredictably
- Constantly force conversation

Core Philosophy

- Stability > features
- Presence > intelligence
- Honest > fake cheerful
- Predictable > flashy
- Local-first > cloud dependence

4. Business Model

This is the cornerstone of Scout's value. Every architectural decision supports the no-subscription principle.

App: $9.99 one-time purchase on Google Play Store. No recurring fees, ever. No subscriptions.

Baseline Brain (Included With Every Scout)

- TinyLlama 1.1B Chat (Q4_K_M, ~669 MB) ships with every Scout via Google Play Asset Delivery
- Runs entirely offline on the user's own device — no cloud dependency
- Every Scout, no matter how the user configures it, has TinyLlama as the foundation
- Verified to run on the Samsung Galaxy A32 at ~12 tokens/sec (Termux test)

Optional Upgrades (One-Time Purchase, No Subscription)

- Larger language models — e.g. Llama 3.2 1B Instruct (~800 MB), Phi-2 (~1.79 GB)
- Delivered via Google Play Asset Delivery (on-demand) — Google's CDN, links never break
- Users who do not upgrade still have a fully working Scout — TinyLlama always works

Optional Gemini Integration

- Users may add their own free Gemini API key in Settings for more advanced online conversation
- Scout NEVER ships with a bundled API key — every user uses their own quota
- Protects Patrick from any shared-quota disaster at scale

- Preserves the no-subscription principle — free Gemini stays free for the user

Why This Matters

- No subscription = no rolling cost to families
- No bundled API key = no shared-quota burnout when Scout scales
- TinyLlama baseline = every user gets a working Scout regardless of upgrades
- Upgrades fund continued development without changing the value of the base product

5. Architecture

Five-Layer Memory Stack

Layer	Type	Storage	Status
Working	Sensory	RAM	Done
Habit	Patterns	JSON 14-day decay	Done
Truth	Authority	SQLite	Done
Relevance	Index	Local Vector	Not yet
Reflective	Wisdom	LLM (read-only)	Not yet

Sovereign Rules

- SQLite Truth always overrules everything else
- Privacy Gate: Gemini receives anonymized text only (planned, not yet implemented)
- Nicolas Protocol: dog detection → immediate stop
- Guest Mode: unknown face → "Hello, I am Scout. What is your name?"

6. Visual & Behavioral Direction

ScoutFaceView is a custom Canvas-based face animation system rendered on the Android phone.

Visual Elements

- Background: dark blue-charcoal #1E2B38 (finalized May 18, 2026)
- Virtual canvas: 1920×1080
- Eyes: large ovals, deep blue iris with 28 spoke rays, biased inward 20f toward nose
- Mouth: minimal subtle curve
- Brows: thin, currently under refinement — floating sticker feeling reduced but still readable
- Wave bars: 22 diamond shapes, teal #00FFD0, visible during listening and speaking
- Idle listening dots: 3 teal pulsing dots when quiet

Animation Philosophy

Desired tone: subtle human animation, soft emotional transitions, calm organic motion, believable presence.
NOT: Pixar-style exaggeration, cartoon expressions, hyperactive motion, fake emotion.

Reference target (May 2026): An AI-generated face video demonstrates the desired calm minimal animation style — gentle gaze drift, very subtle mouth, soft thin brows that integrate with the face. Scout is closer to this target than initial review suggested; brow integration is the largest remaining visual gap.

7. Current Technical State

Working

- Animated face (ScoutFaceView)
- Speech recognition (Android STT) — always listening in PRESENCE mode
- Text to speech (Android TTS)
- Camera — face detection (ML Kit), scene labeling
- Gemini API integration (dev model: gemini-2.5-flash-lite as of May 17, 2026)
- Memory layers: TruthDb, ConversationDb, HabitLayer, PeopleDb, JournalDb
- Intent router — handles time, date, greetings, family facts, downloads, vision, weather
- "My name is Patrick" → stored permanently (teaching system)
- Knowledge downloads — dictionary, WordNet, idioms, sentiment, slang
- Gemini cooldown discipline — real single-flight guard, 429 retryDelay-aware cooldown clock, daily-quota detection with 6-hour cooldown, three-state honest messaging
- Weather — current conditions, tonight, tomorrow, specific day (Mon–Sun), 7-day forecast via Open-Meteo (free, no API key). Cached 30 min for current, 3 hours for forecasts. Falls back to cache when offline. Honest out-of-range message beyond 7 days.

Recently Resolved (May 17, 2026)

The Gemini 429 RESOURCE_EXHAUSTED loop was diagnosed and fixed. Two real bugs:

- geminiRequestInFlight was checked but never set to true — the in-flight guard was dead code
- Google's retryDelay from the 429 response body was ignored entirely, causing Scout to hammer the API

GeminiClient.kt was replaced with a real single-flight guard (AtomicBoolean.compareAndSet) and a cooldown clock that respects Google's retryDelay. Model swapped from gemini-2.5-flash to gemini-2.5-flash-lite for more daily free-tier headroom.

Recently Resolved (May 18–22, 2026)

- Background color finalized at #1E2B38 — two-line edit in ScoutFaceView.kt
- ScoutPromptBuilder.kt extracted from MainActivity — first real reduction of MainActivity size
- Daily-quota detection added to GeminiClient.kt — 429 with PerDay quotaId sets a 6-hour cooldown
- ScoutPromptBuilder.kt updated with three-state messaging — daily quota, short cooldown, generic failure
- ScoutGeminiManager.kt extracted from MainActivity (May 20, 2026) — Gemini orchestration moved into dedicated brain helper. Second successful MainActivity reduction.
- ScoutGeminiManager.kt fully wired into MainActivity (May 22, 2026) — old tryGemini() function and all Gemini state variables removed from MainActivity. Chain is now: handleUnknownIntent →

scoutGeminiManager.tryGemini → GeminiClient.

• ScoutWeatherManager.kt created and wired in (May 22, 2026) — live weather via Open-Meteo, no API key needed. Supports current, tonight, tomorrow, specific day of week, and 7-day forecast. ACCESS_COARSE_LOCATION permission added to manifest and startup request.

• ScoutIntentRouter.kt updated (May 22, 2026) — weather question patterns added including tonight, day names, will-it-rain/snow

Current Gemini Architecture

MainActivity → ScoutGeminiManager → GeminiClient, with ScoutPromptBuilder handling prompt generation separately.

Pending — Ready to Wire In

• ScoutPresenceDecider.kt — fully reviewed, version 3, not yet connected to MainActivity

Known Issues — Code Exists, Not Yet Active

Barge-in (Last reviewed: May 22, 2026)

Barge-in is the ability for the user to speak to Scout while Scout is still talking. Scout would stop mid-sentence and listen immediately.

Code was written and tested but has been deliberately disabled.

Why it was disabled:

Scout's microphone stays active in PRESENCE mode at all times. When Scout speaks through the speaker, the STT (speech recognizer) picks up Scout's own voice and treats it as a user command. Scout then responds to himself. This creates a runaway loop.

What a real fix requires:

• Stop the speech recognizer the instant TTS begins (onStart callback)

• Do not restart the recognizer until TTS has fully finished (onDone callback)

• Add a short buffer delay after onDone before restarting — gives the audio hardware time to clear Scout's voice from the mic

The infrastructure already exists: **isSpeaking** flag, TTS **UtteranceProgressListener** with onStart and onDone, **stopListeningSafe()**, and **scheduleListenRestart()**. The pieces are there. The timing and buffer delay are the unsolved part.

■ Status: Parked. Do not attempt to reactivate without a full session dedicated to this. Reactivating it incorrectly will cause the looping problem to return.

Pending — Future Major Milestones (In Agreed Order)

• 1. MainActivity cleanup / shortening — in progress

• 2. Wire ScoutPresenceDecider.kt

• 3. Offline Brain Phase 1: NDK + llama.cpp .so library compilation

• 4. Offline Brain Phase 2: wire TinyLlama via LlamaEngine + OfflinePromptBuilder + BrainClient

- 5. Offline Brain Phase 3: Google Play Asset Delivery for TinyLlama and optional larger models
- 6. Bluetooth body — KEYESTUDIO Mini Tank Kit V2
- 7. Scout writing his own behavioral code
- 8. Connected Services Phase 1 — Calendar integration
- 9. Connected Services Phase 2 — Phone call awareness
- 10. Connected Services Phase 3 — Gmail read and compose

8. Working Rules

Original Rules — Always Apply

- Full paste-ready replacements only, one file at a time
- No snippets, no partial files
- Stability first, one safe change at a time
- Never touch speech, camera, or download systems without explicit discussion first
- Both Claude and ChatGPT are active collaborators — cross-review welcome
- Upload this Master Project Summary at the start of every new conversation
- Patrick is not a professional programmer — explain at screenshot level
- Patrick is dyslexic, stroke survivor, blind in right eye, type 1 diabetic — keep messages clear, concise, and not visually overwhelming

Amendment (May 17, 2026): Surgical CTRL-F Edits for Large Files

For very large files like MainActivity.kt, full-file replacement carries real risk. A new approved workflow is permitted for small surgical changes:

- Claude provides an exact CTRL-F search string, long enough to be unique in the file
- Patrick searches and confirms exactly one match. If zero or two-plus matches, stop and report
- Claude provides the replacement text
- Patrick triple-clicks the matched line to select it and pastes the replacement

This applies only to small surgical changes. New files and major rewrites still ship as full files.

Amendment (May 22, 2026): Multi-Step CTRL-F Instructions

For edits that span multiple lines or require deleting a block of code, Claude may give detailed multi-step instructions within a single edit block. For example: CTRL-F to find a unique line, then delete from that line down to a closing brace, then paste replacement text directly under. Patrick confirms each full edit block before moving to the next.

9. Key Files

File	Description
MainActivity.kt	Main app — all logic, inner classes, brain
ScoutFaceView.kt	Custom face canvas — all visual animation

GeminiClient.kt	Gemini HTTP wrapper with cooldown discipline (updated May 17, 2026)
ScoutPromptBuilder.kt	Builds Gemini system instruction and online-unavailable messages (extracted May 18, 2
ScoutGeminiManager.kt	Gemini orchestration — fully wired in (May 22, 2026)
ScoutWeatherManager.kt	Live weather via Open-Meteo — current, tonight, tomorrow, day-of-week, 7-day forecast.
ScoutIntentRouter.kt	Intent type routing — handles known questions locally including weather
TeachExtractor.kt	Extracts facts like names from "my name is X" statements
HabitLayer.kt	Pattern memory — standalone reference
LlamaEngine.kt	Offline brain JNI wrapper (written, not yet integrated)
ScoutOfflineBrainGuide.docx	Complete step-by-step integration guide for offline brain
ScoutPresenceDecider.kt	Pending — fully reviewed v3, not yet wired in

10. Settings Architecture

The Settings screen is the user's control center for Scout. Five logical sections, built around the principle that defaults are calm and safe — users opt in to more capability rather than opt out.

10.1 Identity & Voice

- Robot Name — default "Scout", users can rename
- Voice pitch slider
- Voice speed slider
- Future voice tone options

10.2 Brain & Behavior

- Offline Mode (default ON) — Scout uses TinyLlama by default
- Online Brain Helper — toggle Gemini or a larger local model
- API key entry — user's own free Gemini key, never bundled
- Kid Safe Filter
- Pet Safety Protocol — includes Nicolas Protocol enforcement
- Presence Mode (default ON) — Scout actively listens in the room
- Allow Spontaneous Comments
- Privacy Mode toggle

10.3 Builder's Workbench

- Enable Hardware Mode (off by default)
- Bluetooth pairing — for KEYESTUDIO chassis
- Future motor controls

10.4 Privacy & Data

- Memory Export — back up TruthDb and habits
- Memory Import
- Reset Memory Layers — selective reset
- Camera controls
- Voice camera commands

10.5 Extras & Support

- Cosmetics (Backpack) — visual customization
- Support option
- About & Licenses

10.6 Connected Services (Future — All Opt-In)

- Calendar access — add, remove, and announce events. Uses Android Calendar Provider. No external API needed.
- Phone call awareness — Scout announces caller name then steps aside. Normal call behavior untouched.
- Gmail access — read emails and compose when asked. Requires Google OAuth. Planned for a later phase.

Design principle: Scout announces and helps, but never interferes.

11. Hardware Direction — Optional

Scout works fully without hardware. The Builder's Workbench in Settings is opt-in. Reference hardware: KEYESTUDIO Mini Tank Kit V2 (Patrick already owns one).

Relevant Specifications

- Tank-style chassis with motorized treads
- Ultrasonic obstacle avoidance sensors
- Light and ultrasonic follow capabilities
- 8×16 LED panel
- IR remote support
- Communication: Bluetooth

Why Hardware Is Opt-In

- A physically moving robot with weak presence feels emptier than a stationary companion with emotional realism. Hardware comes AFTER personality, emotional realism, local brain, stability, and presence are strong.
- Not every Scout owner wants hardware. The $9.99 baseline must work fully on the phone alone.

12. Future Vision — Scout Writes His Own Behavioral Code

A long-term goal that fits Scout's existing architecture naturally. Scout notices patterns in user behavior, generates a suggestion for a new behavior or routine, and asks permission before activating it. Nothing

changes without the user's explicit consent.

Already Partially Designed In

- Proposal Sandbox layer was built exactly for this purpose
- Approval workflow already exists and is tested through the download system
- LLM (TinyLlama, upgraded model, or Gemini) generates the suggestion text
- TruthDb stores what gets approved permanently

Examples in Practice

- "You always ask about the weather at 7am. Want me to check it automatically each morning?"
- "You seem to talk to me less after 10pm. Want me to use softer responses at night?"
- "You ask about Elijah often on school days. Want me to remember his schedule?"

Important Technical Boundary

Scout cannot write and deploy actual compiled Kotlin code on device. Android does not allow that safely. What Scout CAN do is generate behavioral scripts, response patterns, routing rules, and habit triggers that change his behavior within the existing safe framework. This is real, meaningful self-improvement within boundaries the user controls.

This document is the single source of truth for Project Scout.

Begin every new Claude or ChatGPT conversation by uploading this file.